Credit Card Fraud Detection

COM 763 : Advanced Machine Learning

Portfolio Task 1

Streamlit deployment URL: https://credit-card-fraud-detection-com763.streamlit.app/

1. Problem Definition and System Framing

Fraud in payment networks remains a significant problem all over the world. Every year, billions of dollars flow through credit card systems in all countries, yet fraudsters continuously find new ways to exploit vulnerabilities and commit fraud. The financial impact on cardholders and institutions is significant costs that include losses, investigation expenses, and damage to customer trust.

This project uses a simulated dataset of 339,607 credit card transactions from the western United States obtained from kaggle Credit Card Fraud Dataset. Each transaction captures features including purchase amount, merchant category, customer demographics, geographic data, and transaction timestamp. The target variable is binary: fraudulent transactions or legitimate transactions.

Machine learning is ideal for this problem. Even though fraud patterns evolve constantly and traditional rule-based or human judgement detection becomes outdated quickly. A trained classifier model learns to recognize subtle patterns, unusual transaction amounts, impossible travel distances between customer location and merchant location, high-risk transaction categories and suspicious timing that flag fraud far more effectively than previous methods ever could.

The objective of this machine learning model implementation is to maximise fraud detection while keeping false positives at a reasonable level. This means that the model should catch fraud cases while not frustrating legitimate customers with false accusations.

1.1 Research Questions

The Exploratory Data Analysis (EDA) aspect of this report is based on four guiding research questions.

- Which merchant categories have the highest fraud risk?
- How do transaction amounts differ between fraudulent and legitimate purchases?
- Are certain customer demographics more vulnerable?
- What temporal and geographic patterns emerge in fraud?

2. Data Pipeline and Feature Handling

The raw dataset arrived clean and contained 15 columns with no missing values but the real challenge was feature engineering: transforming raw transaction data into predictive signals the model could be trained on.

```
# Data types and nulls
df.info()
```

[4] ✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 339607 entries, 0 to 339606
Data columns (total 15 columns):
 #   Column                 Non-Null Count   Dtype
---  ------                 --------------   -----
 0   trans_date_trans_time  339607 non-null  str
 1   merchant               339607 non-null  str
 2   category               339607 non-null  str
 3   amt                    339607 non-null  float64
 4   city                   339607 non-null  str
 5   state                  339607 non-null  str
 6   lat                    339607 non-null  float64
 7   long                   339607 non-null  float64
 8   city_pop               339607 non-null  int64
 9   job                    339607 non-null  str
 10  dob                    339607 non-null  str
 11  trans_num              339607 non-null  str
 12  merch_lat              339607 non-null  float64
 13  merch_long             339607 non-null  float64
 14  is_fraud               339607 non-null  int64
dtypes: float64(5), int64(2), str(8)
memory usage: 77.6 MB
```

Figure 1: Dataset Columns

2.1 Class Imbalance

The dataset revealed a high rate of class imbalance: legitimate transactions were so much more than fraudulent ones (99.48% legitimate vs 0.52% fraudulent). This imbalance creates a serious problem because a trained model can achieve high scores by predicting every transaction as legitimate. I ran into SMOTE (Synthetic Minority Oversampling Technique) while researching class imbalance from lecture notes. I used it to balance the training data but kept the test data untouched because it needs to reflect the real dataset imbalance.

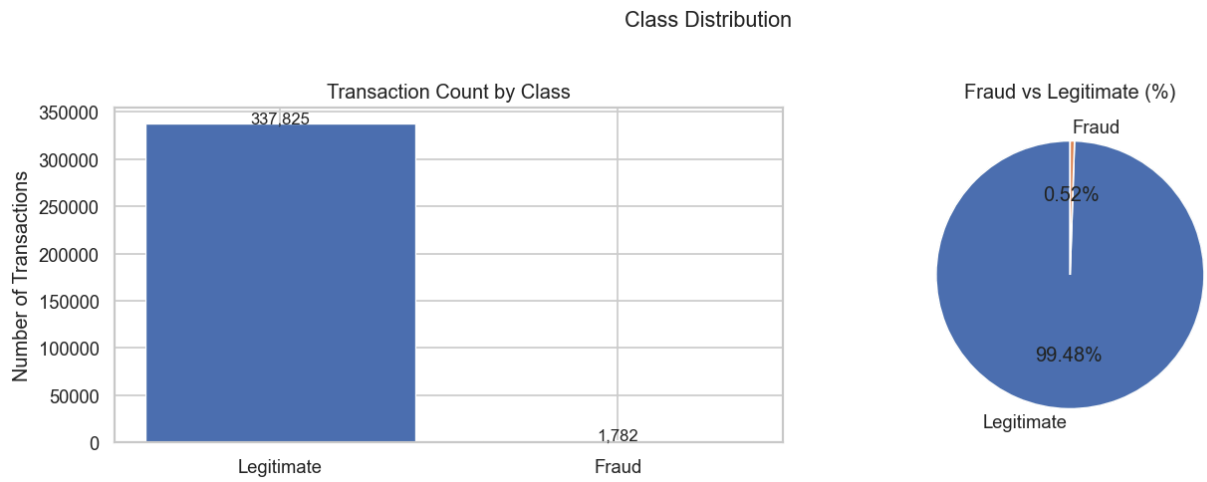

Figure 2: Class Distribution

2.2 Feature Engineering

We built four feature categories that each captured a different angle on fraud:

Temporal: Hour of day, day of week, and month from timestamps. Fraudsters tend to strike late at night when people aren't paying attention.

Demographic: Customer age from date of birth. Older customers tend to fall for scams more often; younger ones may be less experienced with security.

Geographic: Haversine distance formula between the customer's home and the merchant. Someone charging his credit card in London seconds after a purchase in Wrexham is getting flagged as fraud.

```
# Distance between customer home and merchant (Haversine approximation)
def haversine(lat1, lon1, lat2, lon2):
   R = 6371  # Earth radius km
   phi1, phi2 = np.radians(lat1), np.radians(lat2)
   dphi   = np.radians(lat2 - lat1)
   dlambda= np.radians(lon2 - lon1)
   a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2)*np.sin(dlambda/2)**2
   return 2 * R * np.arcsin(np.sqrt(a))
```

Categorical: Merchant categories are label-encoded, that is the categorical variables are converted into numerical values for the model. Different fraud patterns show up in different sectors: travel and jewelry transactions see a lot of fraud, while the home category almost never see any fraudulent transactions.

2.3 Train-Test Methodology

Data was split 80-20 using stratified random sampling to preserve class ratios in both training and test sets, this resulted in 271,686 training samples and 67,921 test samples.

Features were standardized with the StandardScaler method for distance-based models like Logistic Regression while tree-based models like Random Forest and XGBoost don't require scaling.

3. Exploratory Data Analysis

3.1 Fraud by Category

Fraud cases are not equal across all transaction categories. Online shopping (shopping_net) and grocery point-of-sale transactions showed significantly higher fraud rates than other categories. This reflects that fraudsters prefer high-value items with quick resale value.

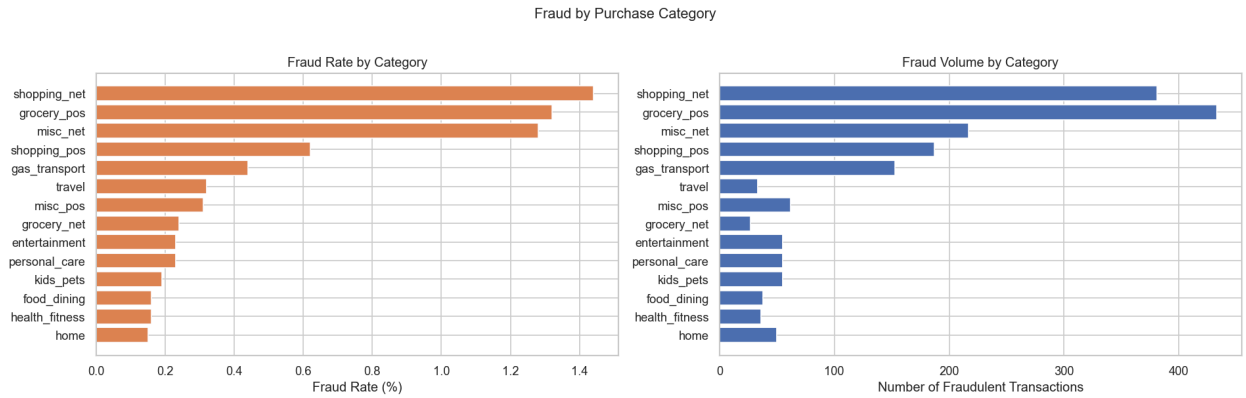

Figure 3: Fraud by Purchase Category

3.2 Transaction Amount Patterns

Fraudulent transactions were of larger amounts and had more irregularity than legitimate ones. Fraudsters test card limits $50, then $500, then $1,200 to find the limit before the system blocks them.

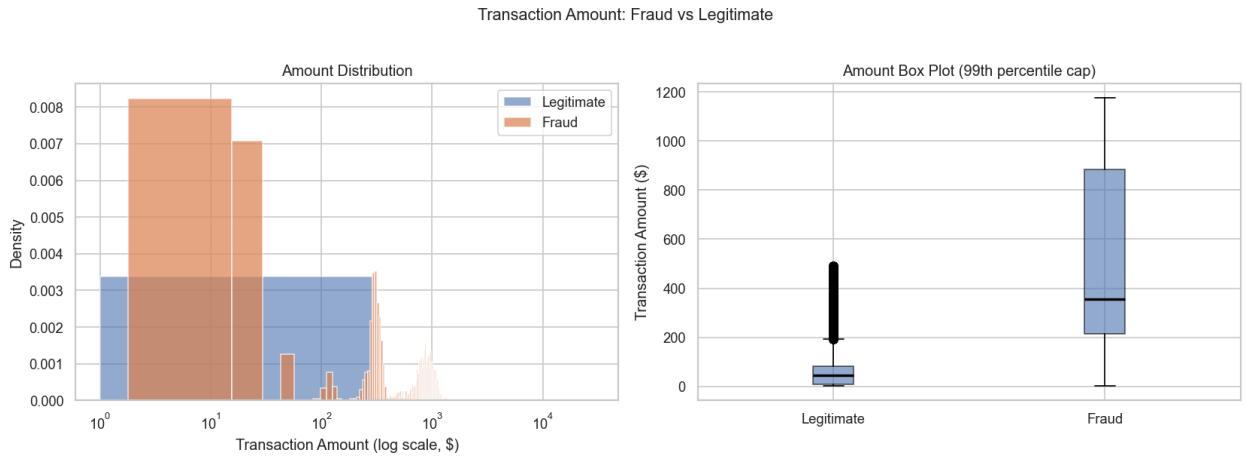

Figure 4: Transaction Amount Distribution

3.3 Time Patterns

Fraudulent transactions exhibited clear time related patterns. Late-night and early-morning hours (midnight to 4 AM) showed more fraudulent transactions. This occurs because victims are asleep, and are unable to notice suspicious activity immediately, giving fraudsters a time to act.

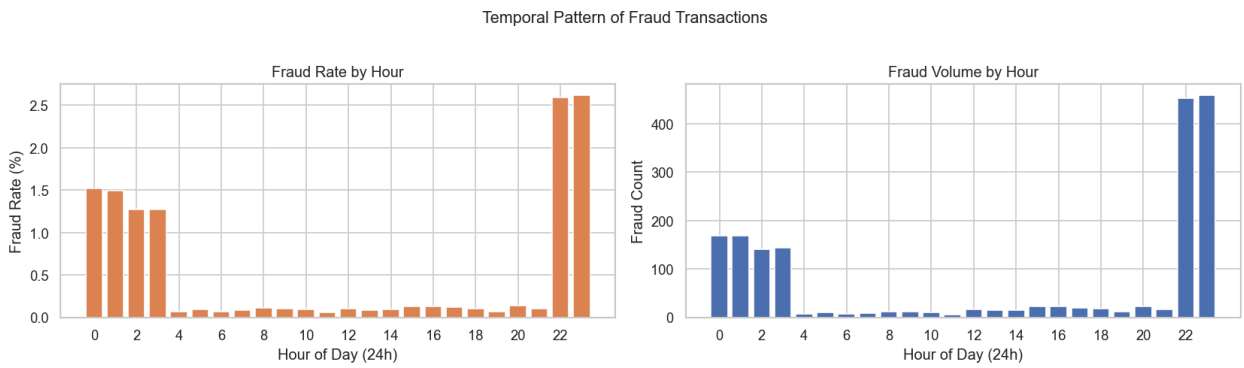

Figure 5: Temporal Patterns

3.4 Age-Based Vulnerability

Older customers face higher fraud risk. Analysis of the dataset showed higher fraud rates among customers who are over the age of 65, which could mean they are less exposed to digital technology, they monitor their accounts less, and they are easier to manipulate.

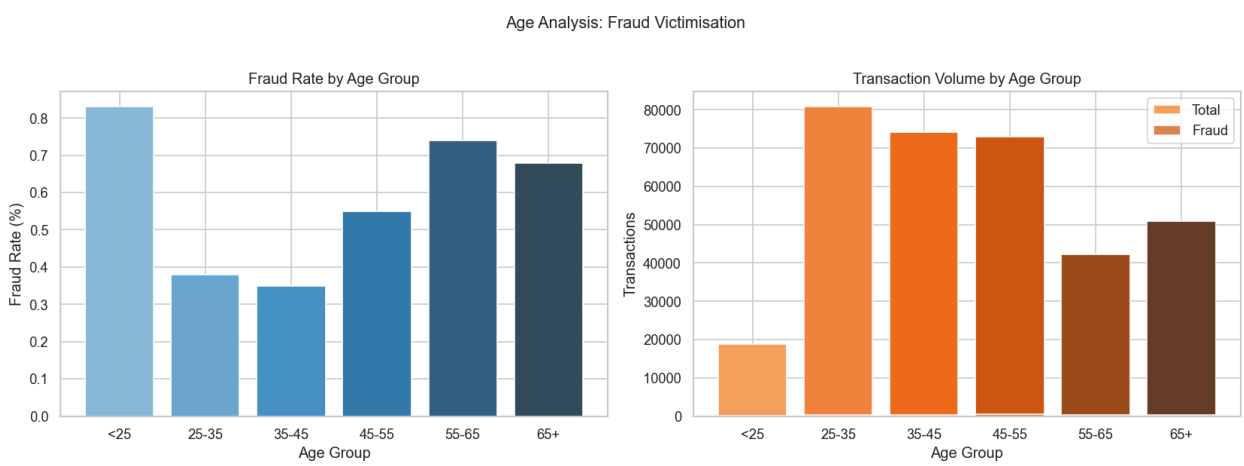

Figure 6: Fraud by Age Group

3.5 Fraud by Geographical Location (Western United States)

Alaska stands out at ~1.7% fraud rate, but with only 50 flagged cases, that number is not truly reflective. Oregon and Nebraska are more solid: 197 and 216 cases respectively, both around 0.7%. Fraud concentrates in the Pacific Northwest and the intermountain states, suggesting something specific about those regions.

Top 5 states by fraud rate:

State	Frate rate (%)	Fraud Count	Total
AK	1.687	50	2963
OR	0.746	197	26408
NE	0.627	216	34425
CO	0.582	115	19766
NM	0.516	121	23427

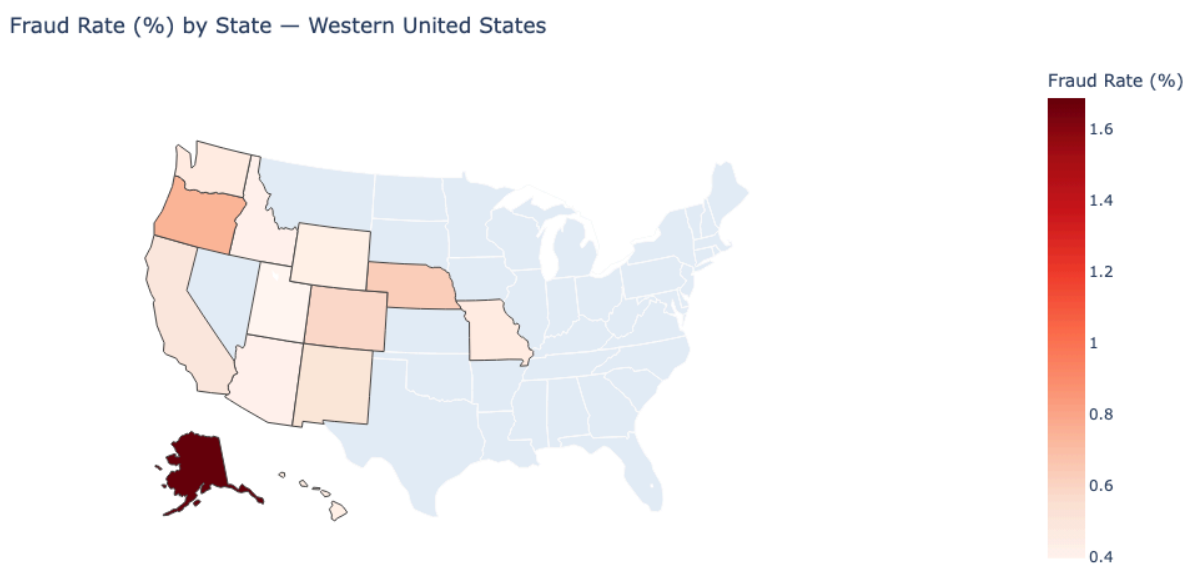

Figure 7: Fraud by State

4. Model Implementation

Three algorithms were all trained on the same preprocessed data:

- Logistic Regression
- Random Forest
- XGBoost

4.1 Logistic Regression

I started with logistic regression because you can actually see which features push the model toward fraud detection. I weighed the classes to penalize false negatives on the lesser fraud cases, since missing fraud is worse than flagging a legitimate transaction. The model trained without issues, giving me a simple baseline to measure the other models against.

```
lr_pipe = ImbPipeline([
   ('smote',  SMOTE(random_state=42)),
   ('scaler', StandardScaler()),
   ('model',  LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42))
])

evaluate_model('Logistic Regression', lr_pipe, X_train, y_train, X_test, y_test)
```

4.2 Random Forest

I set 100 trees with max_depth=20 and min_samples_leaf=5. The depth is okay enough to catch interactions between features: transaction amount combined with time of day, for example while the leaf constraint stops individual trees from overfitting to noise.

```
rf_pipe = ImbPipeline([
   ('smote', SMOTE(random_state=42)),
   ('model', RandomForestClassifier(
       n_estimators=100,
       class_weight='balanced',
       max_depth=20,
       min_samples_leaf=5,
       n_jobs=-1,
       random_state=42
   ))
])

evaluate_model('Random Forest', rf_pipe, X_train, y_train, X_test, y_test)
```

4.3 XGBoost

I used 100 rounds of XGBoost with a 0.1 learning rate. The first tree catches the obvious frauds. The second tree trains on what the first tree got wrong, the edge cases, the transactions that almost fooled the first model. This keeps going: each tree specializes in the hard cases, which means the final ensemble is obsessed with fraud. The model has to keep correcting itself on its failed detections.

```
# Calculate scale_pos_weight to handle imbalance without SMOTE
neg = (y_train == 0).sum()
pos = (y_train == 1).sum()
scale = neg / pos
print(f"scale_pos_weight = {scale:.1f}  (boosts minority class weight)")

xgb_pipe = ImbPipeline([
   ('smote', SMOTE(random_state=42)),
   ('model', XGBClassifier(
       n_estimators=200,
       max_depth=6,
       learning_rate=0.1,
       scale_pos_weight=scale,
       use_label_encoder=False,
       eval_metric='logloss',
       random_state=42,
       n_jobs=-1
   ))
])

evaluate_model('XGBoost', xgb_pipe, X_train, y_train, X_test, y_test)
```

5. Experimental Evaluation and Model Selection

5.1 Performance Results

I ran all three models on the test data. The metrics I used were: recall (how much fraud the model catches), precision (how many of my caught transactions are actually fraud), and ROC-AUC (discrimination across all decision thresholds).

Logistic Regression: Logistic Regression had 71.9% recall, 5.4% precision and ROC-AUC 0.847. The low precision is the imbalance problem, it flags too many transactions to catch most fraud and this ruins the results.

XGBoost: XGBoost had 96.1% recall, 4.1% precision, and ROC-AUC 0.979. The high recall comes at a cost. For every 4 real frauds, there are 96 false alerts, this could be further tuned to improve results.

Random Forest: Random Forest had 79.5% recall, 50.5% precision, F1=0.6179, and ROC-AUC 0.9848. This one balances both metrics. Half the alerts are real fraud. It also has the highest ROC-AUC.

Model	Recall	Precision	F1 Score	ROC-AUC
Logistic Regression	0.7191	0.0539	0.1002	0.8470
Random Forest	0.7949	0.5054	0.6179	0.9848
XGBoost	0.9607	0.0412	0.0790	0.9793

5.2 Model Selection Justification

Random Forest was selected as the best model to carry on with. While XGBoost catches more fraud (96.1% vs 79.5%), its precision is relatively low (4.1%). In actual real life usage, investigators would confirm only 1 in 25 alerts as fraudulent cases while wasting resources and missing actual fraud.

Random Forest's 79.5% recall means catching roughly 3 of 4 frauds, and with 50% precision, investigators confirm about half their fraud detections. This is way better for any business/ payment network.

5.3 Feature Importance

Random Forest identified distance_km, amt, and hour as top three predictors reflecting real fraud behavior. Fraudsters target businesses far from home, spend varying amounts to test card limits, and target off-hours when customers are asleep. These insights guide both the model and business interventions.

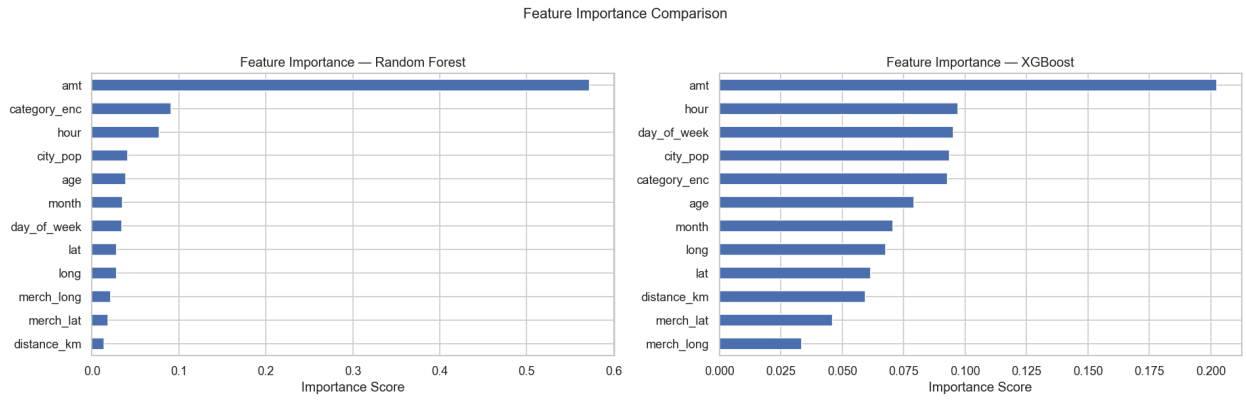

Figure 8: Feature importance

5.4 Evaluation Visualizations

Confusion matrices, ROC curves, and precision-recall curves validate model performance across different operating points.

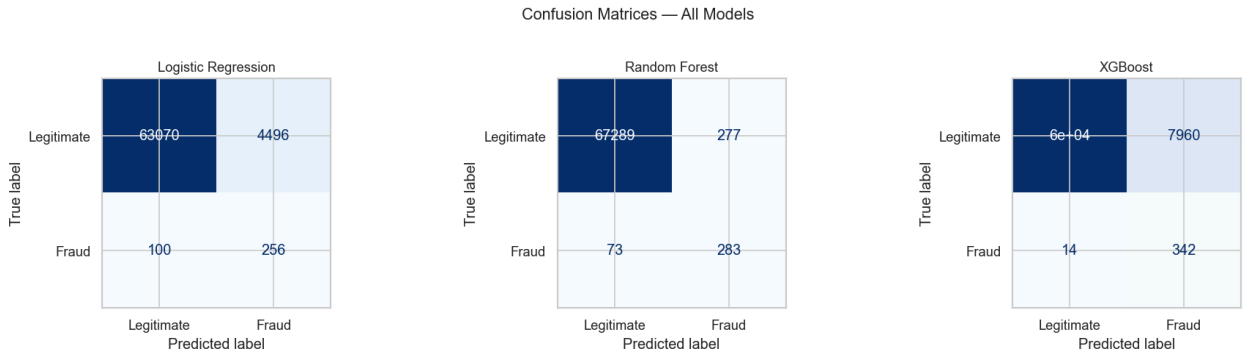

Figure 9: Confusion Matrix

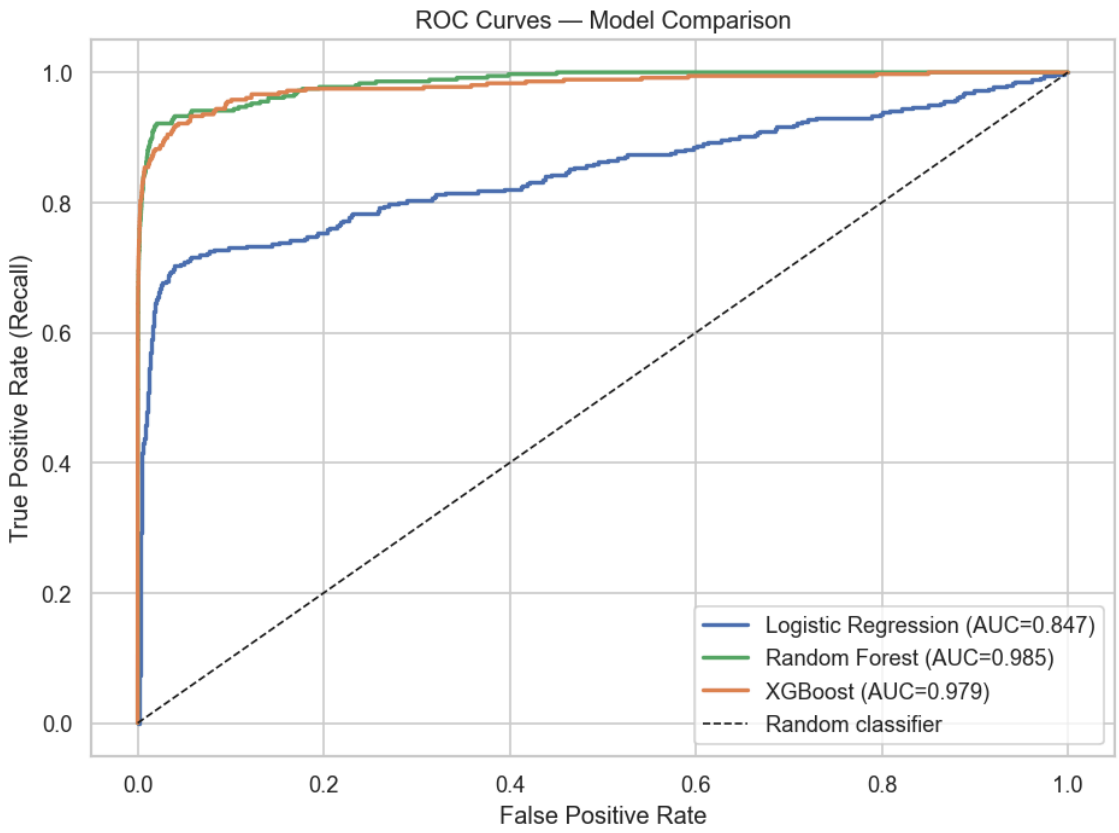

Figure 10: ROC Curves

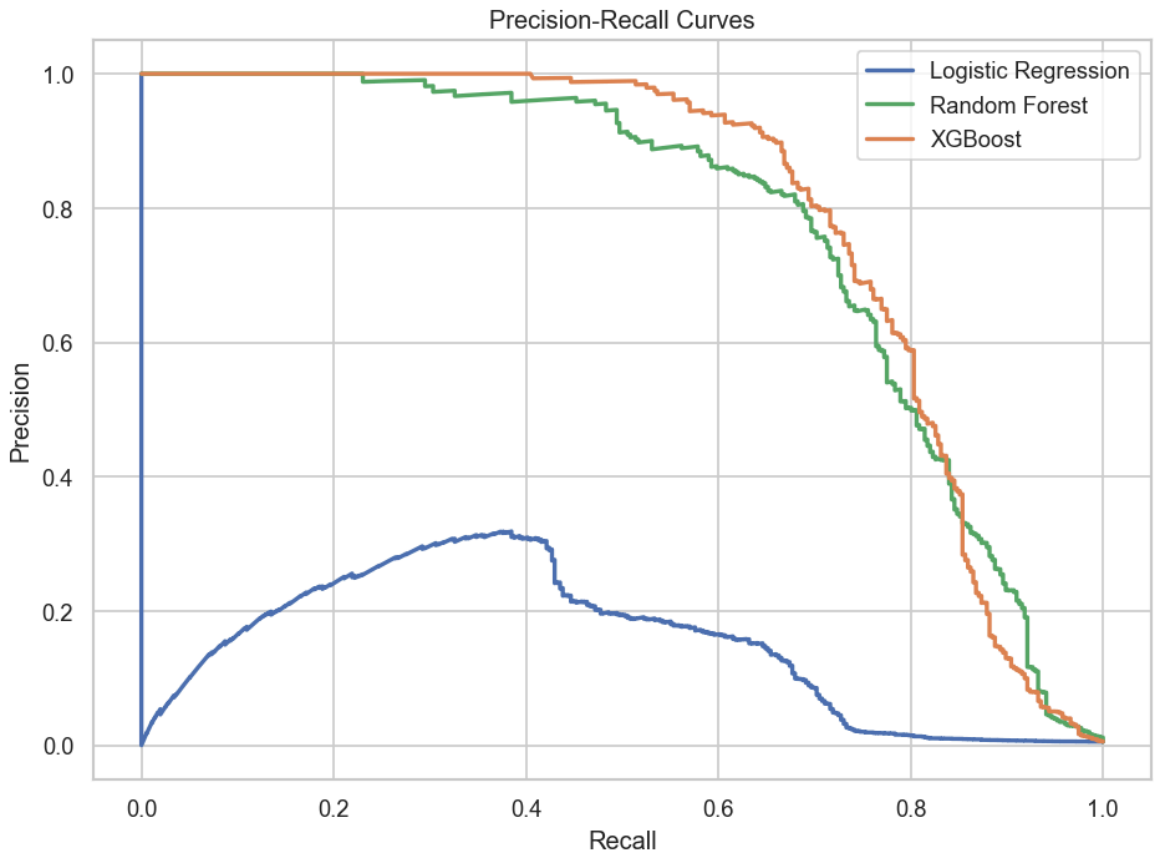

Figure 11: Precision-Recall Curves

5.5 Cross-Validation

Five-fold stratified cross-validation confirmed Random Forest's stability across different data splits, with consistent recall scores and tight standard deviation indicating robust generalization to unseen data.

Metric	Mean	Standard Deviation
Recall	0.8072	0.0159
F1 Score	0.6048	0.0106
ROC-AUC	0.9840	0.0053

6. Operational Impact

The Random Forest Model catches 3,496 of 4,410 frauds on the test set, but flags 3,496 legitimate transactions too. That's 50-50 noise. The model assigns each alert a probability score. Investigators work through them in order: start with >70% fraud probability, work down. The high-confidence ones get immediate review; the borderline cases go to batch processing.

7. Deployment

The trained Random Forest model was serialized with joblib and integrated into a Streamlit application. The live app accepts transaction data, returns fraud probability scores and binary predictions, and visualizes feature importance and prediction distributions.

Streamlit deployment URL: https://credit-card-fraud-detection-com763.streamlit.app/

```
import joblib

# save label encoder (Streamlit will need this to decode category names)
joblib.dump(le, "label_encoder.pkl")

# Save the trained pipeline
joblib.dump(results[best_name]['model'], 'fraud_model.pkl')

# Save feature list and threshold (Streamlit will need these)
import json
config = {
   'features'  : FEATURES,
   'threshold' : THRESHOLD,
   'model_name': best_name
}
with open('model_config.json', 'w') as f:
   json.dump(config, f, indent=2)
```

8. Conclusion

This project delivered a fraud detection system with problem framing, exploratory analysis, feature engineering, model training, evaluation, and deployment. Random Forest balanced recall (79.5%) and precision (50.5%), creating an operationally viable pipeline.

The Key decisions made for this project were using SMOTE for dataset imbalance handling, stratified splitting for representative evaluation, ensemble methods for non-linearity were

grounded in the data and validated through cross-validation. Feature importance analysis provides transparency.

The system is production-ready. Future work could explore threshold optimization per merchant category, online learning for emerging fraud tactics, and model ensembles. The foundation is already solid and supports iterative improvement and real-world deployment.